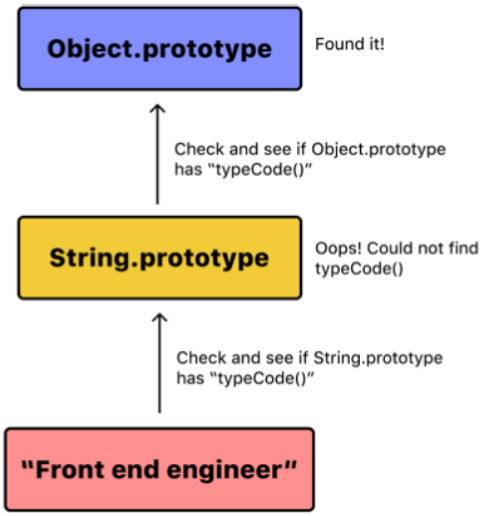
JavaScript - objektové programování

Objekty a prototypy

- **Objekty**:
  - JavaScript je prototypový, objektově orientovaný jazyk
  - Objekty jsou kolekce vlastností (klíč-hodnota)
  - Každý objekt má prototyp, ze kterého dědí vlastnosti a metody
- **Prototypy**
  - Jsou mechanismus pro dědění vlastností a metod mezi objekty
  - Každý objekt má interní odkaz na prototype, který může být využit pro sdílení funkcionality
  - Umožňují vytvořit objekty, které dědí od jiných objektů, aniž by bylo potřeba tříd
  - **Proč**?:
    - JavaScript nepoužívá klasický třídní model s dědičností jako jazyky C++ nebo Java
    - Prototypový model je jednodušší a flexibilnější pro dynamické jazyky
    - Umožňuje dynamicky přidávat nebo měnit vlastnosti a metody objektů
    - **Koncept**:
      - Objekty dědí vlastnosti přes Prototype Chain (řetězec prototypů)
      - Když se přistupuje k vlastnosti objektu, JavaScript nejprve hledá vlastnost v samotném objektu
      - Pokud ji nenajde, pokračuje hledáním v prototypu objektu, a tak dále až k Object.prototype



- **Výhody**:
  - Flexibilita v dědění a sdílení vlastností
  - Úspora paměti díky sdílení metod mezi objekty
  - Dynamické přidávání a změny vlastností
- **Nevýhody**:
  - Může být méně intuitivní pro vývojáře zvyklé na jazyky se standardními třídami
  - Složitost při ladění, pokud je řetězec prototypů hluboký
  - Méně formální struktura může vést k nekonzistencím

- **Vytváření objektů a prototypů:**

Objektový literál

```
const person = {
  name: 'Alice',
  age: 30,
  greet: function: function() {
    console.log('Hello!');
  }
};
```

Konstruktorová funkce

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.greet = function() {
  console.log('Hello!');
};
```

Třídy (ES6) a prototypy

- Syntaxe tříd je syntaktický cukr nad prototypy
- umožňuje jednodušší zápis objektově orientovaného kódu
- Pod pokličkou stále funguje prototypové dědění

Příklad třídy

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  greet() {
    console.log('Hello!');
  }
}

const alice = new Person('Alice', 30);
```

Dál k objektům

- Předávají se referencí
- Když přiřadíme objekt do jiné proměnné, obě proměnné se ukazují na stejný objekt v paměti. Změna přes jednu se projeví ve druhé

```
const a = { name: "Alice" };
const b = a;
b.name = "Bob";
console.log(a.name); // "Bob" — je to stejný objekt
```

Porovnávání objektů:

```
{ name: "A" } === { name: "A" } // false — dvě různé reference
const x = { id: 1 };
const y = x;
x === y; // true — stejná reference
```

Přístup k objektům:

- Pomocí tečky:

```
objectName.propertyName
```

```
person.firstname + " is " + person.age;
```

- Pomocí hranatých závorek:
  - Hodí se když property name je uložena v proměnné

```
objectName["propertyName"]
```

```
person["firstname"] + " is " + person["age"];
```

```
let n1 = "firstName";
let n2 = "lastName";

let name = person[n2] + " " + person[n2];
```

- Změnění property:

```
person.age = 10;
```

- Přidání nové property:

```
person.nationality = "English";
```

- Mazání property:
  - delete smaže oboje i hodnotu i property

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
};

delete person.age;
```

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
};

delete person["age"];
```

- Kontrola jestli property existuje:

```
const person = {
  firstName: "John",
  lastName: "Doe"
};

let result = ("firstName" in person);
```

- Vnořené objekty:

```
myObj = {
  name:"John",
  age:30,
  myCars: {
    car1:"Ford",
    car2:"BMW",
    car3:"Fiat"
  }
}
```

- Objekt a metoda:

```
const person = {
  firstName: "John",
  lastName: "Doe",
  age: 50,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};
```

- S použitím this

```
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  getId: function() {
    return this.id;
  }
};

let number = person.getId();
```

- Jak zobrazit objekty?:
  - Když dáme vypsat celý objekt takto -> let text = person
  - Tak se nám jen vypíše [Object Object] tam kde je očekáván string
  - Správně objekty vypíšeme následovně:
    - let text = person.name + ''‚'' + person.age;
    - for loop:

```
let text = "";
for (let x in person) {
  text += person[x] + " ";
};
```

    - Object.values():

```
const myArray = Object.values(person);

// Stringify the Array
let text = myArray.toString();
```

    - Object.entries():
      - Zlehčuje práci s loops

```
const fruits = {Bananas:300, Oranges:200, Apples:500};

let text = "";
for (let [fruit, value] of Object.entries(fruits)) {
  text += fruit + ": " + value + "<br>";
}
```

OOP

- **Proč**?:
  - Místo několika funkcí a proměnných seskupíme vše do objektů
  - Škálovatelnost
  - Znovupoužitelnost kódu
- **4 pilíře OOP:**
  - **Zapouzdření (Encapsulation):**
    - Zapouzdření znamená, že objektu skryjeme vnitřní informace a zpřístupníme jen nezbytné informace
    - **Hlavní myšlenky:**
      - **Ochrana dat:**
        - Vnitřní stav objektu není přístupný přímo
      - **Kontrola přístupu:**
        - K datům se přistupuje jen přes metody (gettery/settery)
      - **Stabilita objektu:**
        - Uživatel objektu nemůže rozbít jeho vnitřní logiku

```
function BankAccount(accNum, accHolder, balance) {
    let _accNum = accNum;
    let _accHolder = accHolder;
    let _balance = balance;

    function showAccountDetails() {
        console.log(`Account Number: ${_accNum}`);
        console.log(`Account Holder Name: ${_accHolder}`);
        console.log(`Balance: ${_balance}`);
    }

    function deposit(amount) {
        _balance += amount;
        showAccountDetails();
    }

    function withdraw(amount) {
        if (_balance >= amount) {
            _balance -= amount;
            showAccountDetails();
        } else {
            console.log("Insufficient Balance");
        }
    }

    return {
        deposit: deposit,
        withdraw: withdraw
    };
}
```

- **<u>Pomocí closures:</u>**

- **Pomocí tříd:**

```
class BankAccount {
    constructor(accNum, accHolder, balance) {
        this._accNum = accNum;
        this._accHolder = accHolder;
        this._balance = balance;
    }

    showAccountDetails() {
        console.log(`Account Number: ${this._accNum}`);
        console.log(`Account Holder Name: ${this._accHolder}`);
        console.log(`Balance: ${this._balance}`);
    }

    deposit(amount) {
        this._balance += amount;
        this.showAccountDetails();
    }

    withdraw(amount) {
        if (this._balance >= amount) {
            this._balance -= amount;
            this.showAccountDetails();
        } else {
            console.log("Insufficient Balance");
        }
    }
}
```

- **Dědičnost (Inheritance)**
  - Je mechanismus, který umožňuje jednomu objektu nebo třídě získávat vlastnosti a metody od jiné
  - Pomáhá s opakovanou použitelnosti kódu a vytvářením hierarchických vztahů mezi třídami
  - Umožňuje použití vlastností a metod z nadřazené třídy
  - Implementováno použitím **extends** v ES6 třídách
  - Podporuje přetěžování metod v podřízených třídách
  - **Prototype based:**

```
function Animal(name) {
    this.name = name;
}
Animal.prototype.speak = function () {
    console.log(`${this.name} makes a sound.`);
};

// Child constructor function
function Dog(name) {
    Animal.call(this, name); // Inherit properties
}

// Inherit methods from Animal
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog;

// Adding a new method to Dog
Dog.prototype.bark = function () {
    console.log(`${this.name} barks: Woof!`);
};

// Creating an instance
const myDog = new Dog("Buddy");

myDog.speak();
myDog.bark();
```

- **Class based:**

```
class one {
    constructor(name) {
        this.name = name
    }
    speaks() {
        return `my name is ${this.name}`
    }
}
class two extends one {
    constructor(name) {
        super(name)
    }
}
const o = new two('Pranjal')
console.log(o.speaks())
```

- **Polymorfismus (Polymorphism)**
  - poly znamená hodně a morphism znamená transformaci jedné formy do druhé
  - Polymorfismus znamená, že stejná funkce může mít různé podoby a může být volána mnoha způsoby
  - **Polymorfismus funguje hlavně dvěma způsoby:**
    - **Method Overriding (přepisování metod):**
      - Dceřiná třída zdědí metodu z rodičovské třídy
      - Ale může ji přepsat vlastní implementací
      - Při volání metody se použije ta verze, která odpovídá typu objektu

```
class Animal {
    speak() {
        console.log("Animal makes a sound");
    }
}

class Dog extends Animal {
    speak() {
        console.log("Dog barks");
    }
}

class Cat extends Animal {
    speak() {
        console.log("Cat meows");
    }
}
```

- **Method Overloading (simulované):**
  - JavaScript nemá skutečně přetěžování metod jako Java nebo C#, situaci, kdy existuje více metod se stejným názvem, ale s různými argumenty ve stejném rozsahu
  - Ale dá se simulovat:
    - Funkce zkontroluje počet nebo typ předaných argumentů a podle toho provede odlišnou logiku

```
class Calculator {
    add(a, b) {
        if (b === undefined) {
            return a + a;
        }
        return a + b;
    }
}
```

- **Abstrakce (Abstraction)**
  - Lze definovat jako koncept skrývání vnitřní složité logiky objektu a zpřístupnění pouze těch vlastností a funkcí, které jsou pro uživatele důležité
  - **Hlavní myšlenky abstrakce:**
    - **Skrývání složitosti**
      - Implementace je skrytá
    - **Modularita**
      - Kód je organizovaný do znovupoužitelných částí
    - **Bezpečnost**
      - Důležitá data nejsou přístupná přímo, jsou skrytá
    - **Znovupoužitelnost**:
      - Kód lze opakovaně použít v různých aplikacích
  - Často se realizuje pomocí funkcí, tříd a modulů
  - **Funkce:**

```
function a(radius) {
    return Math.PI * radius * radius;
}
console.log(a(5));
```

  - **Objekty a metody:**

```
const car = {
    brand: "Toyota",
    start: function() {
        console.log("Car started");
    }
};

car.start();
```

- **Closures:**

```
function Count() {
    let c1 = 0;
    return {
        inc: function() {
            c1++;
            console.log(c1);
        }
    };
}

const c2 = Count();
c2.inc();
c2.inc();
```

- **Třídy:**

```
class B{
    #balance;
    constructor(B1) {
        this.#balance = B1;
    }
    deposit(amount) {
        this.#balance += amount;
        console.log(`Deposited: $${amount}`);
    }
    getB() {
        return this.#balance;
    }
}

const a1 = new B(1000);
a1.deposit(500);
console.log(a1.getB());
```

Gettery a settery

- **Getter** slouží ke čtení hodnoty
- **Setter** slouží k zápisu hodnoty a může obsahovat validaci
- Definují se klíčovými slovy **get** a **set**

```
class Circle {
  constructor(radius) {
    this._radius = radius;
  }

  get radius() {
    return this._radius;
  }

  set radius(value) {
    if (value < 0) throw new Error('Poloměr nemůže být záporný');
    this._radius = value;
  }

  get area() {
    return Math.PI * this._radius ** 2;
  }
}
```

instanceof a typeof

- **typeof** vrací typ primitivní hodnoty jako řetězec
- `typeof 'ahoj' // 'string'`
- **Instanceof** ověřuje, jestli objekt patří k dané třídě. Prochází celý prototypový řetězec

```
class Person { }
class Student extends Person { }

const bob = new Student();

bob instanceof Student  // true (je Student)
bob instanceof Person   // true (dědí z Person)
bob instanceof Object   // true (všechno dědí z Object)
```